

# GSLT2GSLT: Typed Action Decoding over Verified Interfaces

A machine-checked waist for neural program synthesis, and the unified-carrier architecture  
it licenses

Oruži Zarathustra Goertzel

July 2026 — working draft

## Abstract

We describe an architecture for neural program synthesis over generalized syntax-semantics language theories (GSLTs) in which the interface between learning and logic is machine-checked. The network never emits program text: it scores *typed construction actions* over an exact elaboration state, and a checker owns legality, completion, and acceptance. Three layers are formalized in Lean 4 with kernel-checked proofs: the discrete skeleton (legality masks proved sound, complete, and free of dead ends), a *trust boundary* (learned signals may reorder search but provably cannot lose solutions; hard pruning is licensed exactly by per-property recall certificates, with a machine-checked counterexample showing a plausible uncertified mask destroying a genuine solution), and a *decoder-state theory* (recurrent workspaces as damped fixed-point iteration, with a finite optimal settling depth, a parallel-scan correctness license, and a belief-state cell whose fusion rule is Bayesian conditioning). A single lemma family shows Kalman updating, gated recurrent mixing, and probabilistic-logic evidence revision to be one operation—addition of evidence in natural coordinates, seen from the moment chart. Empirically, the resulting typed-action decoder replicates Alien-Coding-style cumulative self-improvement on OEIS program discovery, with every increment exactly verified; a twelve-cell factorial disentangles decoder architecture from auxiliary supervision; and the same interface, pointed at an external dependent-type benchmark, *measured* the benchmark’s definitional-equality demands ( $\beta\eta$ ) through a graded ladder of verified roots. We state the open comparison the theory sharpens, and assemble its previous alternatives into one unified carrier: a recurrent controller, typed workspace, evidence metadata and routed operator bank share a product state while the checked waist remains unchanged.

**Open artifacts.** The [formal repository](#), [paper source](#), and [compiled PDF](#) are public. Formal links below are immutable source permalinks to commit `84ec10fdac3c`.

## 1 Introduction

Neural program synthesis systems typically trust a language model to emit well-formed programs and filter failures afterwards. We invert the trust: the target language—a GSLT, given as data—induces an exact elaboration state with typed holes; the network only *ranks legal refinements*; the checker computes what each refinement entails and decides acceptance. The comparison to keep in mind is a chess interface on which an illegal move cannot be entered—not because the player was trained to avoid illegal moves, but because the board owns the rules and the move is simply not among the buttons. Here, wrong arity, an out-of-scope variable, or an ill-typed argument is not a low-probability output to be filtered; it is not an action at all. The scientific benefit

is attributability. When the interface between proposer and checker is itself proved correct, a discovery cannot be an artifact of a malformed candidate, and a failure cannot hide an interface bug: three times in the development reported here, an apparent neural failure was traced by the formal layer to an interface boundary (a mask dead-end, a defaulted intermediate type, a missing conversion rule) *before* any training run could mis-attribute it to learning.

The architecture is a waist (Figure 1): one small action vocabulary through which all learning speaks. Everything above the waist (encoders, decoder state, ranking, search order) is free; everything below it (legality, spine construction, acceptance) is checked; and the boundary itself is a pair of theorems rather than a convention.

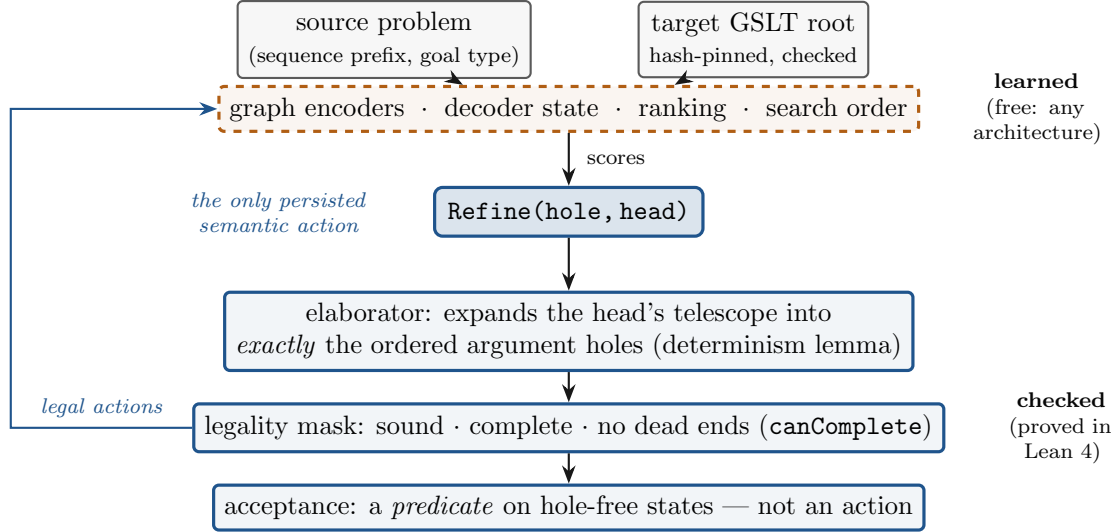


Figure 1: The waist. All learning speaks through one action vocabulary; the checker owns what each action entails and whether a state is accepted. The narrow interface is what makes architecture comparisons attributable: nothing above the waist can widen what is acceptable below it.

## 2 The waist: `Refine(hole, head)`

The persisted semantic action is the pair *refine this typed hole with this head*. Dependent argument holes are not actions: applying a head expands, deterministically, into exactly the ordered argument holes its telescope dictates (a determinism lemma, not a convention). Termination is not an action: acceptance is a predicate on states with no open holes. A policy may factor its score as  $P(\text{hole} \mid s) P(\text{head} \mid s, \text{hole})$ , but the interface persists the pair.

For the first-order root (a 16-operator combinator DSL for integer-sequence programs, after [1]), the legality automaton is specified and sealed in Lean: soundness (accepted streams parse to well-formed programs), completeness (every well-formed program within budget is reachable with every prefix legal), and *no dead ends*—from every reachable legal state some completion exists within budget (`canComplete`). The completion relation is exported with provenance: the operator table and mask fixtures that the Python trainer consumes are generated from the Lean development and hash-pinned, so the learned system and the proved system share one source of truth. The same atomic interface is then instantiated for a dependent typed-hole root (Section 6),

with a proved equivalence to the sealed first-order automaton as the non-vacuity gate: if the generic interface could not express the sealed machine exactly, the interface would be wrong.

### 3 The trust boundary

Two theorems, proved once over the root-parametric interface, fix what learned signals may do.

**Theorem 1** (Soft ranking is always safe). *Any external ranking that lists exactly the legal actions—in any order, from any scorer, at any parameters—preserves the accepted language. Learned guidance can reorder search; it cannot lose or manufacture solutions.*

**Theorem 2** (Hard pruning is licensed exactly by certificates). *A pruning mask derived from a property certified for the target under the evaluation semantics never prunes a state from which a target-reproducing completion exists (*certifiedMask\_recall\_preserving*, lifted from completed programs to partial search states). Sign, parity, and modular-residue certificates instantiate the schema.*

The pair is kept honest by a mandatory counterexample: a plausible, uncertified parity-of-root mask is exhibited, machine-checked, pruning the genuine constant-one solution (*oddRootHardMask\_prunes\_oneSolution*). Uncertified evidence—including everything a world model merely *believes*—receives soft authority only; a world-model hook converts a certified proposition into a licensed mask, and an empty evidence slot fails the premise. This is the design doctrine “the model proposes, the checker disposes” upgraded to mathematics, with the failure mode it prevents exhibited rather than asserted. Figure 2 is the whole boundary on one page.

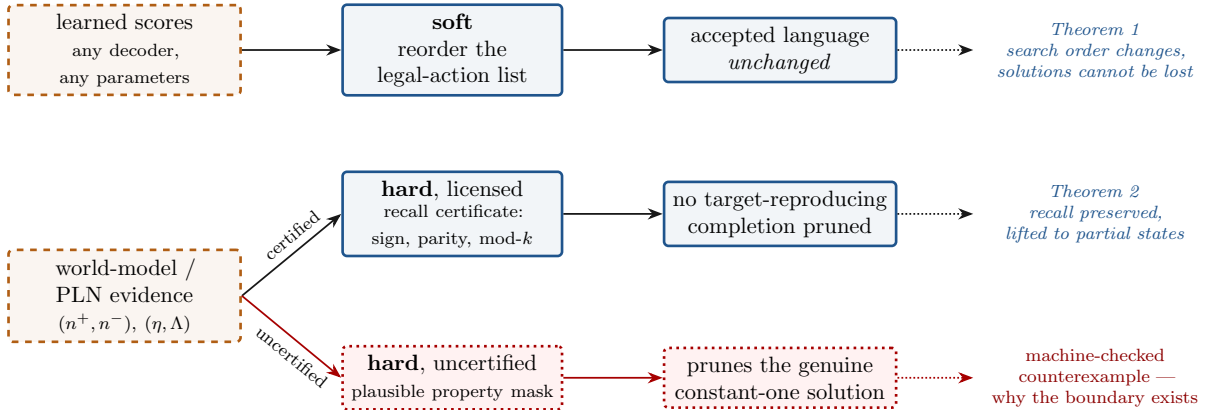


Figure 2: The trust boundary. Learned signals may always reorder; they may exclude only behind a certificate. The red branch is not a caution but a theorem: a plausible uncertified mask provably destroys a real solution.

### 4 Decoder state: from one gated vector to a belief workspace

The incumbent decoder carries construction history in a single gated recurrent vector. Its natural generalization is a *workspace*: slots with fixed addresses and evolving contents, updated by

reusable read–transform–gate–write operators

$$w_i \leftarrow w_i + \frac{1}{K} \sum_{k=1}^K g_{ki} (c_{ki} - w_i), \quad (1)$$

an architecture family we adapt from an unpublished recurrent-workspace design (personal communication, 2026), itself descended from external-memory networks and shared-workspace models [5]. Our adaptation types the addresses: slots are indexed by the typed holes of the elaboration state, so the workspace is a typed register file on which the program graph grows, and slot allocation is total and injective by the sealed open-hole bound.

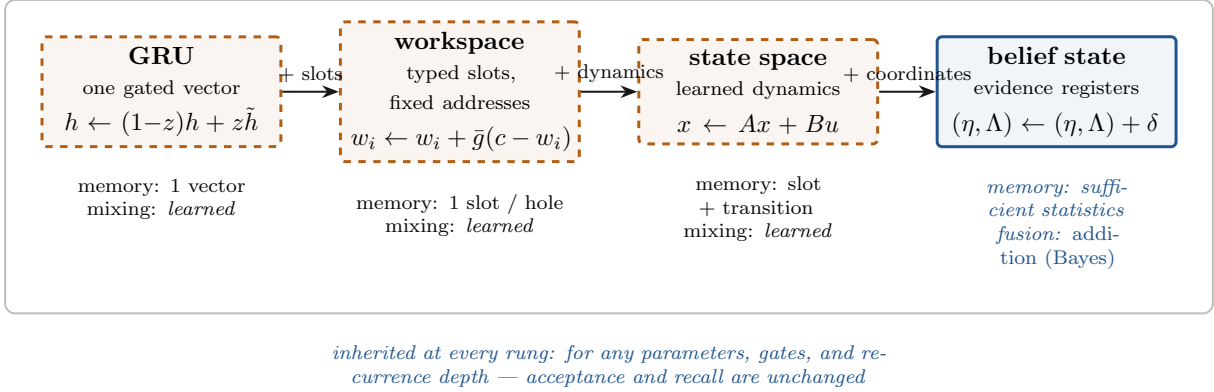


Figure 3: The decoder-state ladder. Each rung adds structure and gives away freedom; only the last replaces learned mixing with the addition Bayes prescribes. The trust boundary holds identically along the whole ladder, so architecture is a free variable.

The formal scope of this family is developed in a dedicated Lean tree (`WorkspaceDecoder`, Figure 3): update (1) is damped fixed-point iteration; under a spectral condition the linear model contracts to a unique equilibrium; and under finite-depth training mismatch there is a *unique fitted recurrence depth*, with over-settling strictly worse—so depth is a convergence parameter with an optimum, not a capacity knob. A prefix-scan theorem (affine transitions form a monoid; scanned prefixes equal the sequential trajectory) is the correctness license for parallelizable linear-recurrent variants [7]. A bridge through the implicit function theorem licenses equilibrium differentiation [6] where the contraction condition holds. Crucially, the trust boundary of Section 3 is inherited: for *any* workspace parameters, gates, and depth, acceptance and recall are unchanged—the entire architecture axis lives on the safe side by construction.

#### 4.1 One machine above the waist: the unified carrier

The four state carriers in the completed factorial are informative as interventions, but they are not four mutually exclusive machine designs. A recurrent vector is a controller; typed slots are memory; belief coordinates are metadata; routed operators are reusable transformations. The unified carrier gives each role its own field while preserving the same GSLT2GSLT interface.

Each allocated slot carries

$$(w_i, n_i^+, n_i^-, u_i^+, u_i^-, t_i),$$

where  $t_i$  is the injective (role, parent, argument) signature and  $u_i^\pm$  records only evidence created in the current episode. Let  $\Lambda_i = n_i^+ + n_i^-$  be the featurewise precision vector and write an overbar

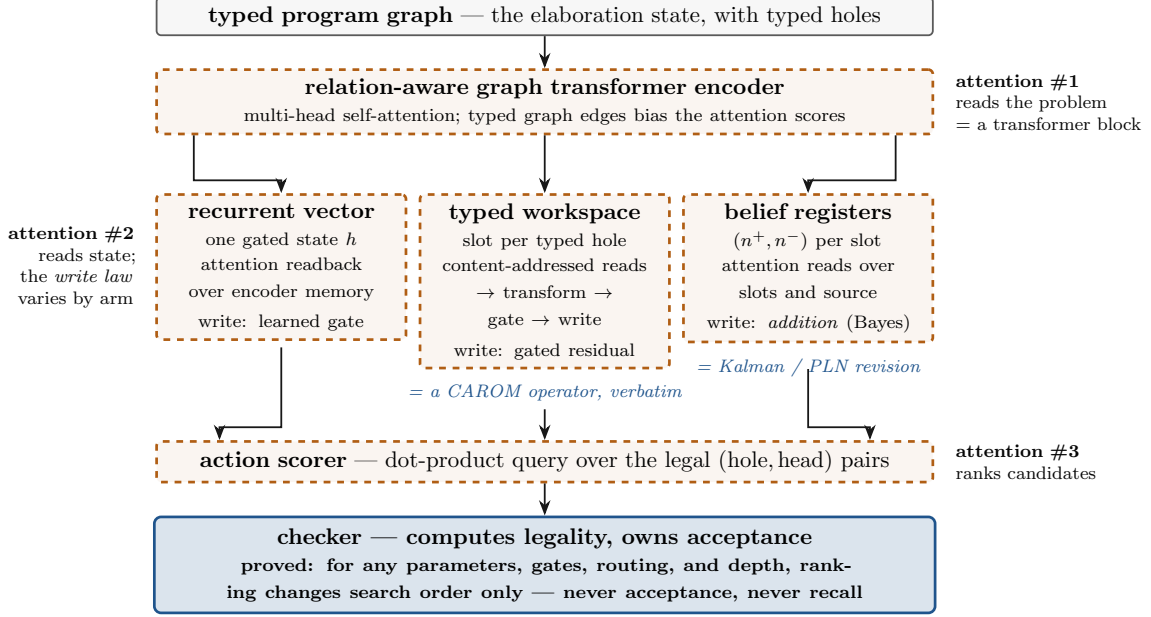


Figure 4: Anatomy of the decoder family, for readers arriving from the transformer world. Attention is the *read* everywhere: the encoder is a standard multi-head transformer block whose scores are biased by the typed graph edges; every state carrier reads its state and the source by content-addressed attention (the workspace’s read–transform–gate–write cycle is exactly a CAROM operator); the action scorer is a single-head dot-product query. The arms differ only in *write* semantics — learned gate, gated residual, or hardwired addition in natural coordinates — which is the decoder-state ladder of Figure 3 seen mechanically. The checker sits below everything: legality and acceptance are computed outside the network, so the entire learned stack can only reorder search.

for its feature mean. A routed operator proposes  $c_{ki}$ , learned write gate  $g_{ki}$  and fresh evidence. The derived update is

$$\begin{aligned}
 r_i &= (1 + \sigma_{\text{jump}}^2 \Lambda_i)^{-1}, & \gamma_i &= \frac{\bar{\Lambda}_i^{\text{fresh}}}{r_i \odot \Lambda_i + \bar{\Lambda}_i^{\text{fresh}}}, \\
 n_i^\pm &\leftarrow r_i n_i^\pm + \text{fresh}_i^\pm, & u_i^\pm &\leftarrow u_i^\pm + \text{fresh}_i^\pm, \\
 w_i &\leftarrow w_i + \frac{1}{K} \sum_k \gamma_i g_{ki} (c_{ki} - w_i), & h &\leftarrow \text{GRU}(x_{\text{carrier}}, h).
 \end{aligned}$$

The operator chooses the candidate and its learned gate; the evidence chart supplies a Bayes-structured damping factor. At zero old precision and positive fresh precision,  $\gamma_i = 1$  and the content step is exactly the typed-workspace update (`contentStep_zeroOld.eq.workspace`).

The read path is coupled without changing the action interface. Slot  $j$  receives

$$\ell_{kj} = \langle q_k, k_j \rangle / \sqrt{d} + b_k^2 \log(1 + \bar{\Lambda}_j),$$

so its unnormalized attention weight is the ordinary content weight multiplied by  $(1 + \bar{\Lambda}_j)^{b_k^2}$ . The identity and its strict monotonicity boundary are machine-checked (`precisionBiasedLogit_strictMono-precision`). Setting  $b_k = 0$  recovers the workspace read exactly.

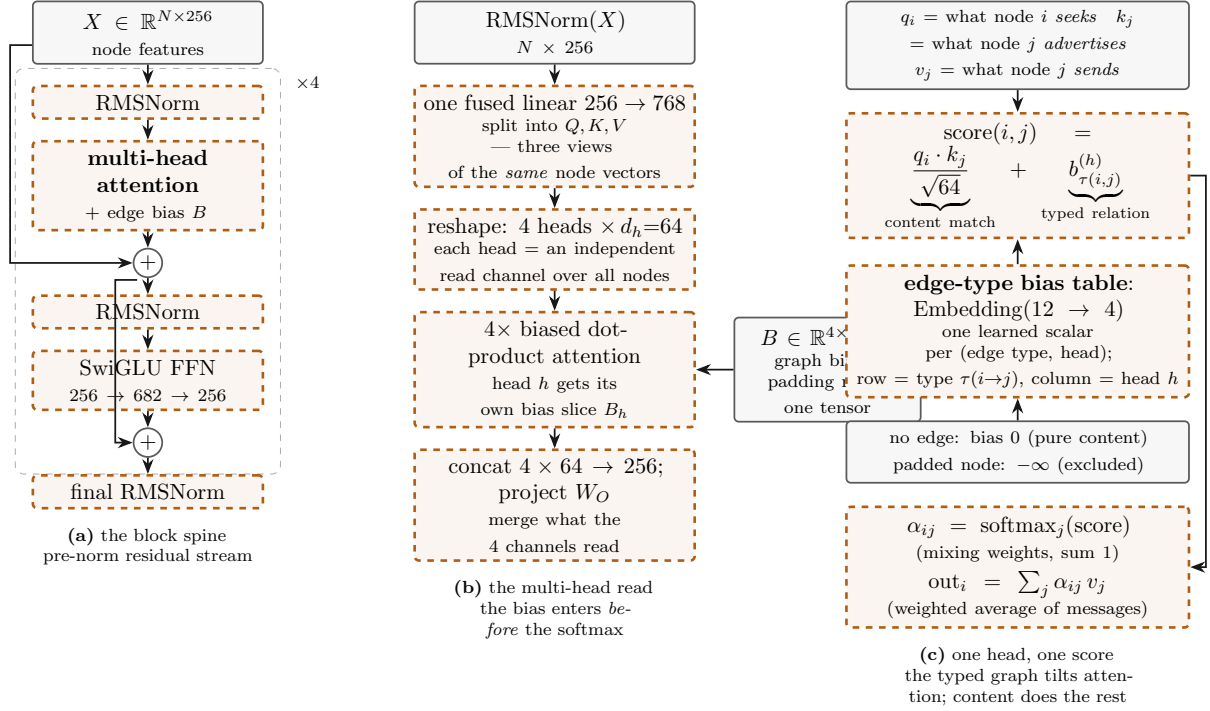


Figure 5: Zooming into the encoder of Figure 4, innermost panel last — self-contained, so the transformer itself can be read off, not just our variant. **(a)** The residual stream: each of four blocks *adds* a refinement to the running node representations rather than replacing them (the  $+$  junctions), with normalization *before* each sublayer (pre-norm; RMS scaling only, no mean subtraction) and a gated position-wise feed-forward (SwiGLU) that processes every node independently. **(b)** Multi-head attention: one linear map makes three views of each node vector — queries, keys, values — split across four independent 64-dimensional read channels; the  $\sqrt{64}$  keeps scores in softmax’s useful range. Rotary position (RoPE) is applied to  $Q, K$  by *node index*, which is meaningful order on the sequence path and incidental on derived/global nodes — graph structure is carried by the bias, not by position. **(c)** The one departure from a vanilla transformer, after Graphormer: each head’s score is content match *plus* a learned scalar for the typed relation between the two nodes, from a  $12 \times 4$  table. No edge means bias zero — the encoder degrades gracefully to a set transformer — and the padding mask rides in the same tensor as  $-\infty$  columns. Everything the model knows about the graph enters through these 48 numbers.

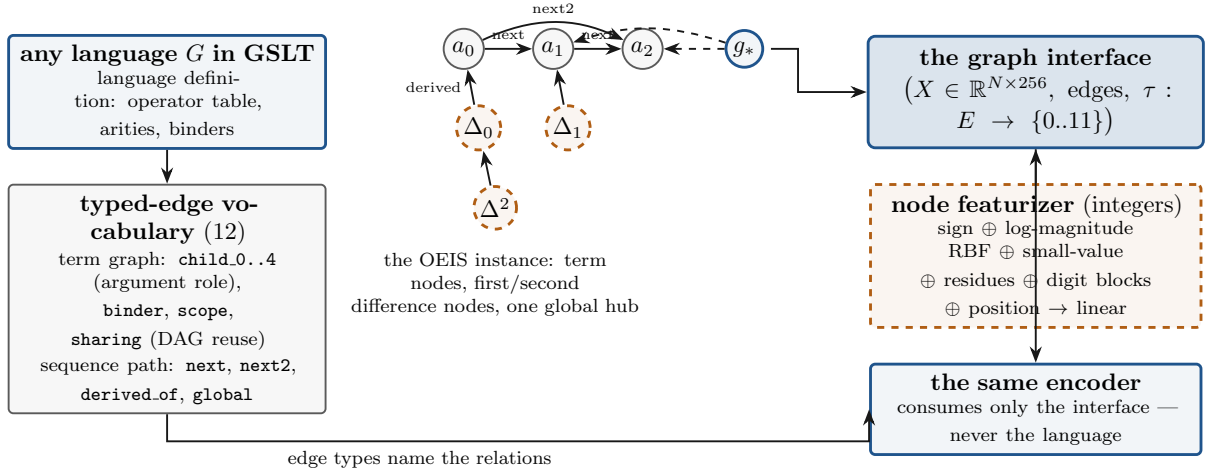


Figure 6: Why the encoder works for any graph  $g$  of any language  $G$  in GSLT. A language definition induces term graphs over a fixed typed-relation vocabulary — argument-role edges **child<sub>k</sub>**, **binder/scope** for variable structure, **sharing** for DAG reuse — and a problem instance contributes its own relations (here: the OEIS sequence path with difference nodes and a global hub). Everything is compiled down to one interface: node feature vectors, an edge list, and an edge-type map. The encoder is defined on that interface alone, so changing the language changes the graph, never the network; legality and typing live in the checker’s mask, outside the learned stack.

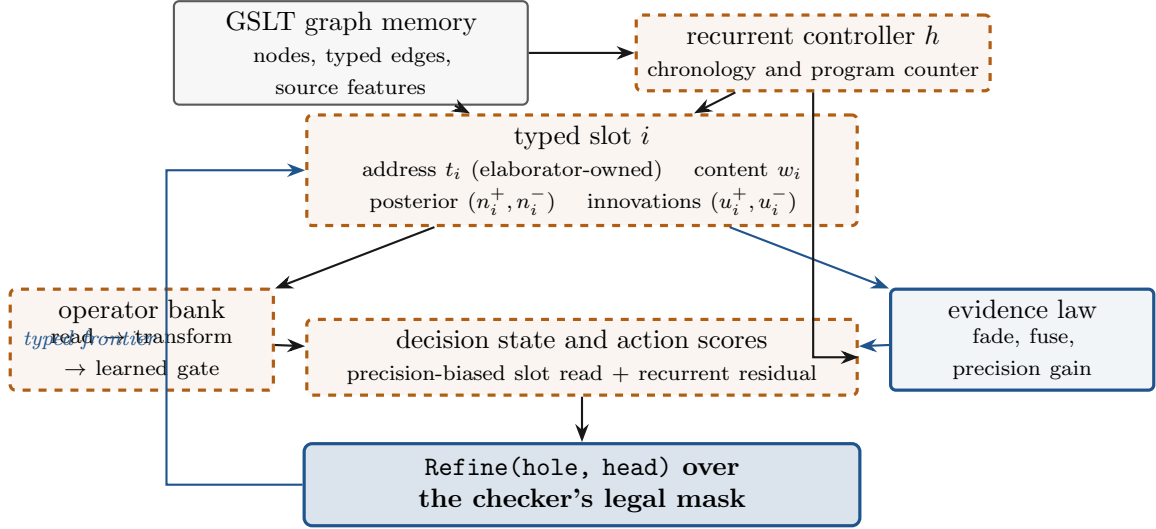


Figure 7: The unified carrier. The address/frontier plane remains checker-owned; all learned recurrence, storage, evidence and routing remains above the same typed-action waist. The architecture composes roles that the factorial separated for attribution.



Persistent evidence is an ordered, optional outer loop. A fresh hole may read a type-indexed prior; at episode completion that table decays once and absorbs only the innovations  $u^\pm$ . It never reabsorbs the posterior  $n^\pm$ , which already contains the imported prior. The double-counting alternative is refuted by an executable fixture (`posterior_reabsorption_doubleCounts_fixture`). Batch absorption is rejected because it would make the order of decay implicit.

This is a refinement, not a relabeling. Under an explicit mode, parameter relation and state projection, every finite policy trajectory equals the typed-workspace trajectory (`workspaceEndpoint_policyTrajectories_eq`). Positive old precision and distinct recurrent histories each supply a strict separating witness, so the full carrier is not definitionally the workspace (`recurrentHistory_fullPolicy_separates`). Most importantly for GSLT2GSLT, the common masked policy contains legal operators and an illegal operator remains absent after a unified step (`illegalOperator_stays_absent_after_unifiedStep`). No carrier state can manufacture a new refinement action.

The reference instance keeps the existing graph encoder, language table, typed frontier, masked scorer and external checker unchanged. Its packed state has  $SW_c + 4SW_e + 2S + W_h + 2$  scalars; the factor four is posterior plus innovation evidence. The exact-real design, an addressed production packet, and one binary32 decision-site replay are in the public formal snapshot. The architecture has not entered the running campaign, so none of the coverage results below are evidence for it. Its first tests must separate recurrent control, precision-read bias, Bayes gating and cross-episode persistence one variable at a time.

## 4.2 Fusion is addition; charts are shadows

The crown of the state theory is a single identity. Write belief in *natural coordinates*: evidence counts  $(n^+, n^-)$  in the discrete case [8], the information form  $(\eta, \Lambda) = (\Sigma^{-1}\mu, \Sigma^{-1})$  in the Gaussian case. Then:

- Bayesian fusion of independent evidence is *addition* in natural coordinates—componentwise for counts, matrix addition for information;
- the Kalman update, the gated-interpolation step (1), and probabilistic-logic revision are the *same* addition, viewed after normalizing to the moment chart; the optimal gate is the Kalman gain, and the workspace equilibrium under exact gains is the Bayesian posterior mean (`equilibrium_iff_eq_posteriorMean`);
- strength and confidence are derived columns of the counts, recovered by the sealed confidence characterization; the counts are the sufficient statistics and the primal state.

The identity is a commuting square (Figure 8): fusion is addition upstairs; every gated-interpolation rule anyone has ever hand-designed is that same addition seen downstairs, after normalizing to the moment chart.

Sequentially, Riccati covariance propagation makes the optimal gain time-varying, and a sharp condition ( $R_2(P + R_1) \neq R_1^2$  in the scalar case) implies every *constant* gate is strictly suboptimal—the derived reason input-conditioned gating (“selectivity”) earns its complexity.

This licenses a concrete cell (Figure 9): slots hold additive evidence registers; learned operators propose *measurements* (what observations mean); a learned gain network, conditioned on the slot’s own precision, scales them; the state update is pure addition; readout charts to moments and scores only legal actions. Everything Bayes determines (how to fuse) is hardwired; everything Bayes leaves open (what observations mean, what the noise is) is learned.



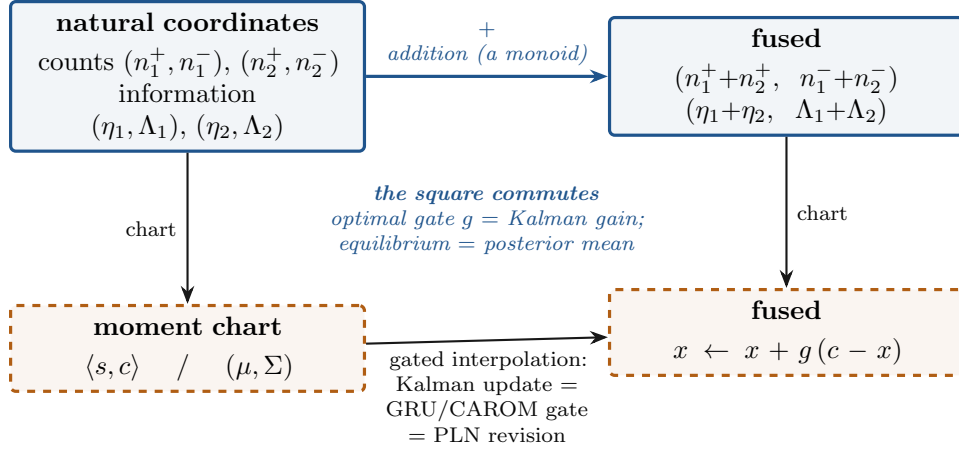


Figure 8: Fusion is addition; charts are shadows. Three rules invented independently in three literatures — Kalman filtering, gated recurrence, probabilistic-logic revision — are one operation, and it is only nonlinear because the moment chart is.

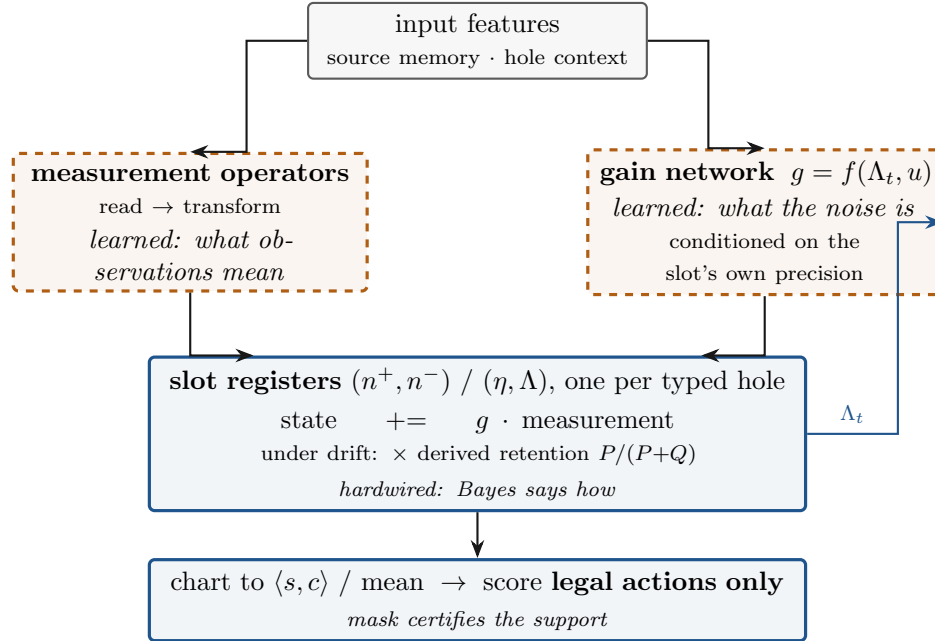


Figure 9: The cell the theorems license. Learning concentrates where Bayes is silent — in the measurement maps and the gain schedule — while fusion itself is the addition Bayes prescribes. The precision register feeds the gain network, which is what makes selectivity possible; the readout can only rank actions the checker already admits.

### 4.3 Where hardwiring is provably wrong: the open world

The optimality of hardwired addition is conditional, and each condition fails somewhere we care about. Addition never forgets: confidence is monotone in accumulated evidence. Under drift—a world whose parameters move—monotone accumulation is *provably* suboptimal; the optimal filter fades memory, and the correct decay rate is derived from the drift statistics, not tuned. Naive addition double-counts correlated re-observation. A single natural-coordinate slot projects away multimodal posteriors. And a miscalibrated measurement model poisons exact fusion, where a learned mixer could have compensated.

We take an explicit position: *optimizing for the closed world is the wrong default*. Worlds drift; open-world probabilistic-logic systems build forgetting in for good reason. Mathematical domains—the OEIS loop below, proof search—are the distinguished near-stationary regimes where hardwired fusion may legitimately excel, and that is precisely why they are the right first battleground: if structure cannot win *there*, it wins nowhere.

The boundary theory is now sealed, and each failure mode is a theorem with a formula. *Nesting*: precision addition realizes exactly the interpolation gates  $[0, 1)$  of the learned-mixing family, and the unit-gate complement splits cleanly into observation discount, strict old-evidence decay, and overwrite—the precise freedoms hardwired fusion lacks. *Double counting*: naive addition overstates precision by exactly the declared evidence overlap, in closed form, and a discounted fusion (proved also count-natively) restores calibration. *Drift*: in the jump model  $P \mapsto P + Q$ , every accumulator whose effective variance stays  $\leq P$  under-gains and carries strictly greater risk whenever  $Q > 0$ ; the derived old-information retention is  $P/(P + Q)$ , moment retention  $R/(P + Q + R)$ , repeated updates fade exponentially, and  $Q = 0$  recovers stationary addition—forgetting as a *derived rate*, not an aesthetic. *Miscalibration*: for a distorted measurement  $y = dx + \varepsilon$ , learned mixing strictly beats the uncorrected unit-model rule exactly off the boundary  $(d - 1)(Pd - R) = 0$ ; but recalibrating the measurement map ( $y \mapsto y/d$ ,  $R \mapsto R/d^2$ ) reproduces the same optimal update with fusion left hardwired—and against the *true*-model gain, learned mixing cannot win at all, since that gain is already optimal. Every fusion-stage repair thus relocates to the measurement map.

The design conclusion is not “hardwire everything” but *learning is necessary, and theorems say where*: in the measurement maps, in the gain schedules, and—under drift—in a decay whose rate the jump statistics determine, while fusion itself remains addition wherever stationarity survives contact with the domain. Each theorem doubles as a pre-registered diagnostic: whether a learned arm’s advantage comes from forgetting, observation discount, dependence repair, or measurement recalibration is identifiable from the per-slot trajectories the experiment schema already carries. One implementation obligation follows: fractional forgetting requires an honest weighted-evidence layer—it is not to be smuggled into natural-number counts.

### 4.4 Routing, order, and the switching boundary

The workspace family has a natural command-driven refinement (personal communication, 2026): a small vocabulary of opaque command tokens, softmax-routed over  $K$  reusable read–transform–gate–write experts, with slot state split into an immutable evidence field, a fixed positional code, and the evolving contents that update (1) governs. Four boundary theorems now frame what routing can and cannot add.

*Routing stays on the ladder*. A simplex mixture of gated-interpolation experts is again a single gated interpolation under the mixed gain (`mixedExpertStep_eq_singleInterpolation`),

so routing buys input-conditioned selectivity—which the Riccati argument above already priced as necessary—without leaving the update class; and the immutable evidence field survives every routing schedule, temperature, and depth (`runTripleSlotSchedule_evidence`). Command tokens themselves carry no semantics: trajectories are determined by the induced routing schedule alone (`commandTrajectory_eq_of_mixtureSchedule_eq`), and distinct tokens with equal routes are behaviorally indistinguishable. Meaning lives in routes, not names.

*Compositionality is commutation.* When are two command phases order-independent, so that a schedule is a *set* of edits rather than a brittle sequence? For affine phases the answer is exact: swapping the order shifts the state by  $[B, A]x + (Ba + b - Ab - a)$  (`affineCompositionDiscrepancy_exact`), and order-independence holds iff the linear parts commute *and* the affine obstruction vanishes (`affinePhases_orderIndependent_iff`); commuting linear parts alone are not enough—a one-dimensional bias counterexample closes that loophole. Equivalently, the pairwise interference energy  $\|[A, B]\|_F^2$  must vanish (`linearPhases_orderIndependent_iff_interferenceEnergy_zero`)—a Gram diagnostic computable from any trained operator pair. Compositionality thereby stops being an emergent hope and becomes a measured certificate.

*Stability does not compose.* Two commands, each individually extinguishing—either alone reaches rest in two steps—alternate to  $4^n$  growth (`individuallyStable_switchingDiverges`). Per-command guarantees are therefore worthless for command *programs*; the honest certificate is schedule-uniform. A common quadratic Lyapunov function bounds every schedule at one geometric rate, and commuting stable generators always admit one (`commutingCommonQuadraticLyapunov_crown`). A command-driven decoder should ship with a switching certificate, not a per-command one.

*Safety is indifferent to all of it.* The trust boundary extends verbatim: for arbitrary routes, temperatures, schedules, and depths, ranked acceptance coincides with checker acceptance and recall of accepted programs is unchanged (`inheritanceCrown`). The routing axis, like the state axis, lives entirely on the safe side; the theorems above are about competence, not safety.

When trained operators cannot be made to commute, there is a second, older route to order-independence: canonicalize. Interleave the learned step with a projection onto canonical forms,  $s_{t+1} = \text{canon}(f_\phi(s_t, c_t))$ , and order-sensitivity is quotiented away rather than trained away: a confluent canonicalizer sends both orders of a critical pair to one normal form—exactly the commutation the raw operators lack—while the per-step projection stops representational drift from compounding across a schedule. The iterated form is a weight-tied loop of the same transformer, the regime in which looped transformers act as programmable computers [11]; canonicalization is the projection that keeps the loop’s state on the quotient. This program already runs on that principle at two other scales: the determinism lemma of Section 2 is the confluence statement for spine creation, and canonical-forms  $LF$  makes conversion vanish into the syntax. The design conclusion repeats itself one level up: learn the proposal, hardwire the quotient.

## 5 The loop compounds, and the factorial that kept us honest

The empirical instance is cumulative self-improvement on OEIS program discovery [1] (Figure 10): train on the accumulated verified corpus, search under fixed budgets, check every candidate against the whole encyclopedia, add what verifies, repeat. The same wake-sleep shape drives other self-improving synthesis systems, which additionally distil reusable abstractions into the library between rounds [10]; here the checker plays the role of the sleep-phase filter, and the

waist fixes what a reusable action is.

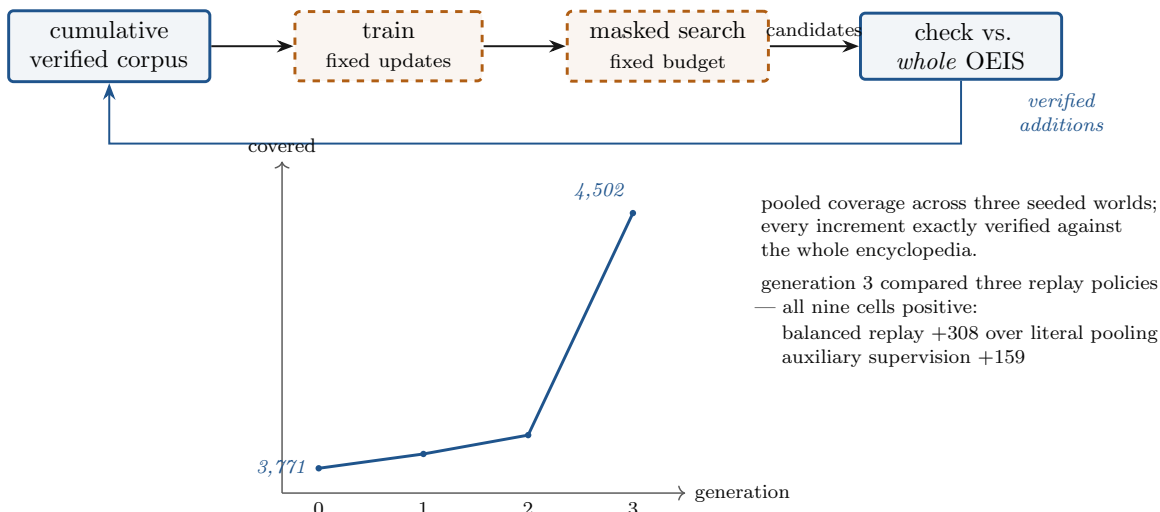


Figure 10: The loop compounds. At literal pooling the model’s own discoveries carry 0.24% of sampling mass and the loop starves on the signal it exists to generate; balanced replay is what makes self-improvement visible.

Three independently seeded worlds grew coverage from a shared initial 3,771 known sequences to 3,986 after two generations, and—under a prospectively registered three-way comparison of replay policies at generation three—to 4,502 before generation four, with all nine cells positive. Balanced replay of the model’s own discoveries (25% mass) beat literal pooling by +308 pooled sequences: at natural sampling mass (0.24%), the loop starves on exactly the signal it exists to generate. Auxiliary predictive supervision added +159. The union of generation-three discoveries was 503 distinct sequences, for 697 verified model-origin programs accumulated by generation four.

Two registered negatives bound the claims. An ablation asking whether a world’s own discoveries beat an equal mass of *unused external* certified data answers no—but that comparator does not exist in the deployed regime, where the corpus is whatever the loop has verified; it is an oracle-advantaged ablation, not the loop objective. And no arm, in any generation, solved its own assigned held-out target within budget: the evidence concerns learned *discovery* guidance over the catalogue, not target-conditioned synthesis.

A twelve-cell factorial (decoder architecture  $\times$  auxiliary supervision  $\times$  three seeds) disentangled an earlier confound: a one-seed comparison had suggested a token-sequence decoder beat the typed-action decoder 38–28; the factorial showed the apparent gap was carried by asymmetric auxiliary supervision, and across seeds the typed-action decoder led pooled (188 vs. 119, positive in 2/3 seeds) with the supervision interaction inconsistent—no interaction claim survives. One-seed architecture stories, in this regime, are noise until replicated.

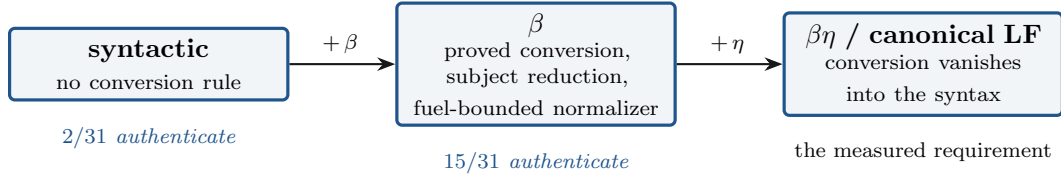
**The completed carrier campaign.** The comparison was subsequently run to its registered eight-generation endpoint in two independently initialized worlds, with four matched-capacity carriers over the same waist. All 64 cells passed independent verification. Their union contained 984 distinct new sequences:

| carrier                   | distinct | exclusive | relative training cost |
|---------------------------|----------|-----------|------------------------|
| recurrent vector (GRU)    | 435      | 241       | 1.00×                  |
| typed workspace           | 383      | 171       | 2.85×                  |
| belief, derived retention | 294      | 119       | 7.94×                  |
| belief, learned retention | 244      | 138       | 3.53×                  |

The result argues for composition rather than a winner-take-all carrier choice: 669 of 984 sequences were found by exactly one carrier, with pairwise overlaps only 8–16%. Yet a fixed-split portfolio reached 399 sequences against the recurrent arm’s 435, so complementarity alone does not license naive allocation. A later generation-9/10 carrier-credit factorial is still running; no partial cell, preliminary coverage count or mechanism trace from it is included in this paper’s empirical totals. The unified carrier of Section 4.1 is a post-campaign design and likewise has no efficacy result here.

## 6 The same waist, pointed outward

Because the interface is root-parametric, pointing it at a new language is an instantiation, not a rewrite. Pointed at an external dependent-type inhabitation benchmark [2], the authentication gate became a measuring instrument for the benchmark itself (Figure 11):



*the 16 residuals have unequal  $\beta$ -normal forms but equal  $\beta\eta$ -normal forms — so checking this benchmark’s witnesses requires full  $\beta\eta$  definitional equality, a fact its own selection criteria do not state. Discovered because the gate refused to blur syntactic equality into conversion.*

Figure 11: A graded ladder of verified roots turns the authentication gate into an instrument: each rung is a checker of strictly greater conversion strength, and the witnesses that fail measure exactly what the benchmark assumes. The ladder doubles as an ablation axis — one policy, evaluated under checkers of increasing power.

under a conversion-free root, 2/31 reference witnesses authenticate; adding proved  $\beta$ -conversion (with subject reduction and a fuel-bounded normalizer whose exhaustion is a classified verdict, never acceptance) raises this to 15/31; the sixteen residuals have unequal  $\beta$ -normal forms but equal  $\beta\eta$ -normal forms—so checking the benchmark’s witnesses requires full  $\beta\eta$  definitional equality, a fact its own “no definitional reduction” selection criterion does not reveal, discovered because the gate refused to blur syntactic equality into conversion. All 31 statements round-trip through the typed interface, and 782 named corruption fixtures (including 596 binder-capture cases) are rejected. The graded ladder of verified roots—syntactic  $\subset \beta \subset$  canonical-forms  $LF$ —doubles as an ablation axis unavailable elsewhere: the same policy can be evaluated under checkers of increasing conversion strength.

A frozen one-shot under the pre-conversion root recorded the honest baseline: a ranker trained only on independent synthetic data solved 0/31 within budget while simple controls solved up to

3 and a native solver [2] solved 31/31—the root and action representation, not neural capacity, are the binding constraint before conversion lands, and the result is banked as the pre-conversion floor for the ladder. Beyond dependent types, a read-only audit of the Metamath database family found thousands of non-copied proofs across related databases, making database-to-database transfer a measurable next rung; and a generated-calculus benchmark waits, gated, behind its checker’s readiness theorem.

## 7 Exact configuration, for replication

The figures above give the architecture in schematic form; this section pins every number a reimplementer needs. All values are read from the reference source and cross-checked against the hash-pinned conformance fixtures.

**Encoder.** Model width  $d = 256$ ; four pre-norm residual blocks, each  $\text{RMSNorm}(\varepsilon = 10^{-6}) \rightarrow \text{multi-head attention} + \text{edge bias} \rightarrow \text{RMSNorm} \rightarrow \text{SwiGLU}$ , closed by a final  $\text{RMSNorm}$ . Four heads of width 64; one fused projection  $256 \rightarrow 768$  supplies  $Q, K, V$ . Rotary embeddings (base  $10^4$ ) are applied to  $Q, K$  by node index, active at the even head width 64 and self-skipping at odd widths. The pre-softmax score is

$$\text{score}(i, j) = \frac{q_i \cdot k_j}{\sqrt{64}} + b_{\tau(i, j)}^{(h)},$$

the edge bias a learned scalar per (edge type, head) from  $\text{Embedding}(12 \rightarrow 4)$  — forty-eight numbers, added per ordered pair, last write winning on a conflicting pair, zero where no edge exists,  $-\infty$  for a masked key. The feed-forward inner width is  $\lfloor 8d/3 \rfloor = 682$ . The OEIS adapter emits four of twelve declared edge roles (`next`, `next2`, `derived_of`, `global`); ids 4–11 (`child_0..4`, `binder`, `scope`, `sharing`) are reserved capacity for other source languages. Each node’s integer feature vector is

| Feature                     | dim                   | Definition                                               |
|-----------------------------|-----------------------|----------------------------------------------------------|
| sign                        | 3                     | one-hot $\{-, 0, +\}$                                    |
| log-magnitude RBF           | 47                    | radial bases over $\text{range}(0, 94, 2)$ on $\log  x $ |
| small-value one-hot         | 66                    | exact bins for $-16 \dots 48$                            |
| residues                    | 71                    | one-hots mod $\{2, 3, 4, 5, 6, 7, 8, 9, 11, 16\}$        |
| digit blocks                | 16                    | eight blocks, base $10^4$ , embedded                     |
| position                    | 16                    | node-index embedding (max 32)                            |
| concat $\rightarrow$ Linear | $219 \rightarrow 256$ | 187 scalar +16 digit +16 position                        |

**State carriers.** The four experimental carriers differ only in write semantics; the read over encoder memory and the checker interface are shared. A capacity test holds all four within two percent of the recurrent baseline. The unified carrier is a fifth, post-campaign design that composes the roles rather than entering the historical comparison.

| Carrier                 | State                  | Write update                                           | Configuration                                                               |
|-------------------------|------------------------|--------------------------------------------------------|-----------------------------------------------------------------------------|
| recurrent (GRU)         | $h$                    | $h \leftarrow (1 - z)h + z\tilde{h}$                   | GRU(256, 256), one layer; unrolls the action stream                         |
| typed workspace         | slots $w_i$            | $w_i \leftarrow w_i + \overline{g_{ki}(c_{ki} - w_i)}$ | 60 slots $\times$ width 112, 4 operators, depth 1                           |
| belief (derived)        | $(n^+, n^-)$           | $n^+ \leftarrow r n^+ + \text{fresh}^+$                | 60 slots $\times$ 96 features;<br>$r = 1/(1 + 0.1 \text{ prec})$            |
| belief (learned)        | $(n^+, n^-)$           | $n^+ \leftarrow r n^+ + \text{fresh}^+$                | same,<br>$r = \sigma(\text{head}(\text{chart}, \text{control}))$            |
| unified (post-campaign) | $(w, n^\pm, u^\pm, h)$ | content gate $\times$ precision gain; innovations add  | 60 slots $\times$ content 108 $\times$ evidence 16; control 64; 4 operators |

The workspace carrier uses fixed pre-order slot addresses that are never reused and per-operator (untied) transforms; the derived-belief update is fade-then-fuse, which equals precision interpolation in natural coordinates. The unified reference instance has 5,323,264 parameters against the recurrent baseline’s 5,238,392, a ratio 1.0162; this is a capacity check, not a performance result.

**Typed-action decoder.** The action set is the sixteen-operator DSL. The scorer forms logits = readout([hidden, context])  $\cdot$  op-embeddings + bias, with context attention over encoder memory scaled  $1/\sqrt{256}$ , then a softmax over the elaborator’s legal-action mask (illegal operators set to  $-\infty$ ). The elaborator, not the network, computes the open-hole frontier from operator arity and child roles; applying an action replaces one hole with its children’s fresh addresses under the invariant actions used + open holes  $\leq 60$ . Intermediate type  $\sigma$  is threaded through child roles {root, value, code}, and the typed-hole vector is role + parent + argument + depth embeddings.

**Parameter counts.** The reference sequence-to-sequence baseline is a 10,781,697-parameter scaled-Luong bidirectional LSTM. The four graph-transformer carriers are held within two percent of one another by test; their absolute total is recorded only relationally, not as a fixed number.

**Training and search budget.** The longitudinal campaign settings, shared by every arm:

| Group                | Setting                                                              |
|----------------------|----------------------------------------------------------------------|
| optimizer            | AdamW, learning rate $3 \times 10^{-4}$ , on the trained parameters  |
| schedule             | 2500 updates, batch 16, gradient clip 5.0, checkpoint every 100      |
| draws per generation | 30000 base + 10000 discovery, shuffled                               |
| search per cell      | beam 16, $\leq 4096$ transitions, $\leq 256$ checker calls           |
| retention search     | beam 16, $\leq 2048$ transitions, budget 64                          |
| seeding              | train seed = seed + generation $\times 100000$ ; worlds hash-derived |

The seed corpus is 3,771 known A-numbers; a new solve is a distinct A-number absent from it covered by an exact program, decided by the external whole-OEIS checker over a fixed train/test split. The predictive-coding update rules layered on this configuration are specified in the companion cost analysis.



## 8 Related work and provenance

The proposer/checker split with learned guidance descends from neural theorem proving and self-learning synthesis [1]; typed-hole elaboration states from live programming semantics [3] and bidirectional elaboration [4]; the workspace family from shared-workspace and external-memory architectures [5] and equilibrium models [6]; selective state-space models [7] supply the parallelizable linear-recurrent variants our scan theorem licenses; evidence-count semantics from probabilistic logic networks [8]; the external dependent-type benchmark and solver from [2]. The contribution here is the machine-checked *waist*: masks, trust boundary, and decoder-state theory proved in Lean 4 with kernel-checked axioms confined to `{propext, Classical.choice, Quot.sound}`, fixtures generated from the proofs, and the learned system bound to them by hash-pinned parity — so that architecture comparisons are attributable, and failures indict the component that actually failed. Complete formal sources are in the immutable [GSLT tree](#), [workspace tree](#), and [unified-carrier tree](#).

## Status

Sealed and machine-checked: the first-order skeleton and its exports; the atomic interface with two proved root instances; the trust-boundary pair with its counterexample; the workspace scope (contraction, fitted depth, scan license, IFT bridge); the sequential and matrix belief theory; the counts-primal unification; the selective belief-decoder contract and its pinned schema; and the fusion boundary theory of Section 4.3 (expressivity nesting, overlap calibration, nonstationary fading, distortion factorization—four lifts, fifteen new theorem families, five explicit scope boundaries, kernel-computed accounting). Also sealed are the unified carrier’s exact-real workspace refinement, strict separation fixtures, retention-economics threshold, addressed-packet witness and binary32 decision-site certificate. The eight-generation, four-carrier comparison is complete; its 64 cells and the 984-sequence union are independently checked. The generation-9/10 credit-rule extension is running and is not a result of this snapshot. The unified carrier is implemented and tested but has not entered a registered efficacy campaign. All coverage numbers above are exact whole-corpus verifications from immutable, independently recomputed ledgers.

## References

- [1] T. Gauthier and J. Urban. *Learning Program Synthesis for Integer Sequences from Scratch*. AAAI 2023; arXiv:2202.11908.
- [2] C. Norman and J. Avigad. *Implementing Dependent Type Theory Inhabitation and Unification*. arXiv:2603.01463, 2026.
- [3] C. Omar, I. Voysey, M. Hilton, J. Aldrich, and M. Hammer. *Hazelnut: A Bidirectionally Typed Structure Editor Calculus*. POPL 2017.
- [4] L. de Moura, J. Avigad, S. Kong, and C. Roux. *Elaboration in Dependent Type Theory*. arXiv:1505.04324, 2015.
- [5] A. Goyal et al. *Coordination Among Neural Modules Through a Shared Global Workspace*. ICLR 2022; arXiv:2103.01197.

- [6] S. Bai, J. Z. Kolter, and V. Koltun. *Deep Equilibrium Models*. NeurIPS 2019.
- [7] A. Gu and T. Dao. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. arXiv:2312.00752, 2023.
- [8] B. Goertzel, M. Iklé, I. Goertzel, and A. Heljakka. *Probabilistic Logic Networks*. Springer, 2009.
- [9] K. Cho et al. *Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation*. EMNLP 2014.
- [10] K. Ellis et al. *DreamCoder: Bootstrapping Inductive Program Synthesis with Wake–Sleep Library Learning*. PLDI 2021; arXiv:2006.08381.
- [11] A. Giannou et al. *Looped Transformers as Programmable Computers*. ICML 2023; arXiv:2301.13196.